This level of customization isn’t often necessary, but it’s quite useful to have the capability to customize your app’s URLs when you need it. In the next section we’ll look at how routing works in practice in ASP.NET Core.

6.2 Endpoint routing in ASP.NET Core

In this section I describe how endpoint routing works in ASP.NET Core, specifically with respect to minimal APIs and the middleware pipeline. In chapter 14 you’ll learn how routing is used with Razor Pages and the ASP.NET Core MVC framework.

Routing has been part of ASP.NET Core since its inception, but it has been through some big changes. In ASP.NET Core 2.0 and 2.1, routing was restricted to Razor Pages and the ASP.NET Core MVC framework. There was no dedicated routing middleware in the middleware pipeline; routing happened only within Razor Pages or MVC components.

Unfortunately, restricting routing to the MVC and Razor Pages infrastructure made some things a bit messy. Some cross-cutting concerns, such as authorization, were restricted to the MVC infrastructure and were hard to use from other middleware in your application. That restriction caused inevitable duplication, which wasn’t ideal.

ASP.NET Core 3.0 introduced a new routing system: *endpoint routing*. Endpoint routing makes the routing system a more fundamental feature of ASP.NET Core and no longer ties it to the MVC infrastructure. Now Razor Pages, MVC, and other middleware can all use the same routing system. .NET 7 continues to use the same endpoint routing system, which is integral to the minimal API functionality that was introduced in .NET 6.